



GUI-Actor: 无坐标视觉定位 面向 GUI 智能体

Qianhui Wu^{*1} Kanzhi Cheng^{*2} Rui Yang^{*3} Chaoyun Zhang¹ Jianwei Yang¹
Huiqiang Jiang¹ Jian Mu² Baolin Peng¹ Bo Qiao¹ Reuben Tan¹ Si Qin¹
Lars Liden¹
Qingwei Lin¹ Huan Zhang³ Tong Zhang³ Jianbing Zhang² Dongmei Zhang¹
Jianfeng Gao^{†1}

¹Microsoft ²Nanjing University ³University of Illinois Urbana-Champaign

Abstract

构建由 VLM 驱动的 GUI 智能体时，一个主要挑战是视觉定位，即根据视觉内容和文本计划，为动作执行定位合适的屏幕区域。大多数已有工作把它表述为基于文本的坐标生成任务。然而，这类方法存在若干限制：由于缺少显式空间监督，空间-语义对齐较弱；由于单点预测会惩罚有效变化，难以处理有歧义的监督目标；并且屏幕坐标是密集的，而 Vision Transformer 等模型提取的视觉特征是粗粒度 patch 级特征，两者之间存在不匹配。本文提出 GUI-Actor，一种基于 VLM 的无坐标 GUI 定位方法。GUI-Actor 的核心是一个注意力式动作头，它学习将专用的 <ACTOR> token 与所有相关视觉 patch token 对齐，使模型能够在单次前向传播中提出一个或多个动作区域。与此相配合，我们进一步设计了一个定位验证器，用于评估候选区域，并从中选择最可能用于动作执行的区域。大量实验表明，在多个 GUI 动作定位基准上，GUI-Actor 优于已有最新方法，并且对未见过的屏幕分辨率和布局具有更好的泛化能力。值得注意的是，GUI-Actor-7B 以 Qwen2-VL 和 Qwen2.5-VL 为骨干时，在 ScreenSpot-Pro 上分别取得 40.7 和 44.6 的分数，超过 UI-TARS-72B (38.1)，同时参数量和训练数据显著更少。此外，通过引入验证器，我们发现只微调新引入的动作头（7B 模型约 ~100M 参数）并冻结 VLM 骨干，就足以达到与已有最新模型相近的性能。这说明 GUI-Actor 能够在不损害底层 VLM 通用能力的情况下，为其赋予有效的定位能力。项目页面：<https://aka.ms/GUI-Actor>。

^{*}共同贡献：qianhuiwu@microsoft.com, chengkz@smail.nju.edu.cn, ry21@illinois.edu.

[†]项目领导。

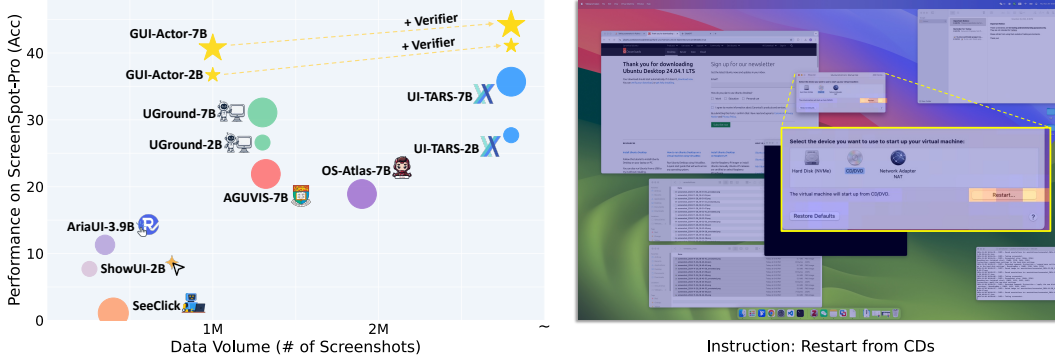


图 1: 左: ScreenSpot-Pro 基准上模型性能与训练数据规模的关系。越靠上且越靠左越好; 点越大表示模型参数越多。为公平比较, 这里只展示基于 Qwen2-VL 构建的 GUI-Actor 模型。以 Qwen2.5-VL 为骨干时, GUI-Actor-3B/7B 的分数最高可达 42.2/44.6 (不使用验证器)。右: 动作注意力示意图。GUI-Actor 通过关注最相关的视觉区域来定位目标元素。

1 引言

随着大语言模型 (LLM) 和视觉语言模型 (VLM) 的快速发展, 研究者越来越关注构建 GUI (Graphical User Interface) 智能体, 使其能够理解自然语言指令, 并在桌面端 [1, 2]、移动设备 [3] 和 Web 应用 [4] 等不同平台的软件界面中自主交互。有效的 GUI 智能体需要两项核心能力: (i) 多模态感知, 用于理解视觉线索和语言线索; (ii) 动作执行, 用于通过鼠标、键盘或触摸屏与数字环境交互 [5, 6]。早期系统依赖结构化元数据 (*e.g.* HTML、DOM 树或视图层级) [1], 但这类数据在不同平台上往往噪声较多、不一致, 甚至不可用。因此, 近期工作更强调视觉 GUI 智能体, 即直接从渲染后的截图中感知界面, 类似人类用户的交互方式 [7]。在这一范式中, 一个核心挑战是视觉定位: 将自然语言计划映射到屏幕区域。大多数已有方法把它视为坐标生成任务, 通过 LLM 使用的同一套文本生成机制输出屏幕位置 (*e.g.* “ $x=0.125, y=0.23$ ”) [8]。

然而, 用坐标生成来表示 GUI 动作, 也就是让模型以文本 token 形式输出屏幕位置 (*e.g.*, $x=\dots, y=\dots$), 会带来几个内在限制。首先, 空间-语义对齐较弱: 生成离散坐标 token 要求模型通过语言建模头, 把视觉输入隐式映射成数字输出, 却没有明确的空间归纳偏置。这一过程效率低、数据需求大, 并且由于缺少直接把视觉特征连接到动作位置的监督, 容易出错。其次, 监督信号具有歧义: 许多 GUI 动作, 例如点击按钮内部, 允许一组有效目标位置。然而, 基于坐标的方法通常把任务当作单点预测, 惩罚所有偏离, 包括合理偏离, 因此无法刻画人类交互中的自然歧义。最后, 视觉特征与动作空间之间存在粒度不匹配: 坐标是连续且高分辨率的, 而 Vision Transformer (ViT) [9] 等视觉模型基于 patch 级特征工作。这种不匹配迫使模型从粗粒度视觉 token 中推断密集的像素级动作, 从而削弱其对多样屏幕布局和分辨率的泛化能力。

尽管一些近期方法 [10] 尝试通过预测边界框而非单点来丰富空间定位, 它们仍然把这些区域表示为原始坐标字符串 (*e.g.* $x_{\min}, y_{\min}, x_{\max}, y_{\max}$), 这些字符串与视觉

特征本身是脱节的。如果没有 ROI pooling [11] 或空间注意力机制 [12] 等架构组件，这类方法仍难以弥合语言意图与视觉动作定位之间的差距。

重新审视人类如何与数字界面交互，可以得到一个关键观察：人类在行动前并不会计算精确屏幕坐标，而是感知目标元素并直接与其交互。受此启发，我们提出 GUI-Actor：一种在 VLM 上增加注意力式动作头的模型，用于实现更接近人类行为的无坐标视觉定位。不同于把动作定位视为坐标预测任务的已有方法，GUI-Actor 学习直接关注相关视觉区域，而不依赖数值坐标生成。GUI-Actor 的核心是一个专用的 <ACTOR> token，它通过联合处理视觉输入和自然语言指令来编码定位上下文。随后，注意力机制学习将该 token 与截图中的视觉 patch token 对齐，从而找到最相关的 GUI 区域。得到的注意力图可以自然地识别界面中的可操作区域。

为了处理 GUI 交互中的固有歧义，即一个 UI 元素 (*e.g.* 一个按钮) 内部可能有多个点都是有效的，GUI-Actor 使用多 patch 监督进行训练。所有与真实边界框重叠的视觉 patch 都标为正例，其余 patch 标为负例。这种监督策略让模型能够容忍空间歧义，并减少对合理动作变化的过度惩罚。此外，由于 GUI-Actor 直接在视觉骨干的原生空间分辨率上定位动作，它避免了已有方法中的粒度不匹配问题，并能更稳健地泛化到不同屏幕尺寸、分辨率和布局。最后，为了支持决策细化，我们进一步为 GUI-Actor 引入一个轻量级定位验证器，用于评估多个候选区域，并选择最可能用于动作执行的区域。

我们的贡献可以概括如下：

1. 我们重新审视了近期用于 GUI 智能体视觉定位的坐标生成方法，指出其局限，包括空间-语义对齐较弱、监督目标有歧义以及特征粒度不匹配，并提出 GUI-Actor，一种能够有效缓解这些问题的新型无坐标方法。
2. 我们设计了一个注意力式动作头，它可以在单次前向传播中生成多个候选区域，从而为搜索策略等下游模块提供灵活性。
3. 我们引入一个定位验证器，用于从动作注意力图提出的候选区域中选择最可能的动作区域。我们还展示了该验证器可以很容易地接入其他定位方法，并进一步提升性能。
4. 大量实验表明，在多个 GUI 动作定位基准上，GUI-Actor 优于在相近数据规模上训练的最新方法，并且对未见过的屏幕尺寸和分辨率表现出更强鲁棒性。值得注意的是，GUI-Actor 的 2B 版本甚至超过了若干 7B 级竞争模型。此外，借助验证器，采用轻量训练的 GUI-Actor (*i.e.* 冻结骨干 LLM，仅微调动作头中新引入的约 $\sim 100\text{M}$ 参数) 可以有效赋予底层 VLM 定位能力，同时不损害其通用能力。

2 相关工作

由 LLM/VLM 驱动的 GUI 智能体。 LLM 和 VLM 的出现推动了 GUI 智能体的发展，使其能够理解自然语言指令，并在移动端 [13]、Web [14, 15] 和桌面环境 [16, 1, 2] 中执行复杂任务。早期研究关注自主智能体框架 [4, 17, 18]，这些框架提示商业模型通

过代码生成 [19, 20] 或工具调用 [21, 22] 与操作系统交互。随着开源且可定制智能体的需求上升, 另一条工作线聚焦于训练 LLM/VLM 来增强其智能体能力, 包括 GUI 理解、规划和执行 [7, 10, 23, 24]。这些工作的关键在于收集 GUI 专用训练数据, 例如 OCR 标注 [25]、界面摘要 [26]、问答对 [23] 和大规模任务演示 [15, 27–32]。

智能体开发的一个核心需求, 是能够与部署在虚拟机和基于 Chrome 的浏览器中的真实 GUI 环境交互。早期智能体操作 HTML 或可访问性树等结构化元数据 [14, 33], 但这类表示在不同平台上脆弱且不一致 [34, 7]。因此, 近期趋势转向以视觉为中心的范式, 即智能体像人一样, 通过原始截图感知界面, 并使用鼠标和键盘输入进行交互 [35, 36]。在这一设定下, 一个核心挑战随之出现: 把自然语言指令定位到具体 GUI 区域, 本文称之为 *GUI 视觉定位*。

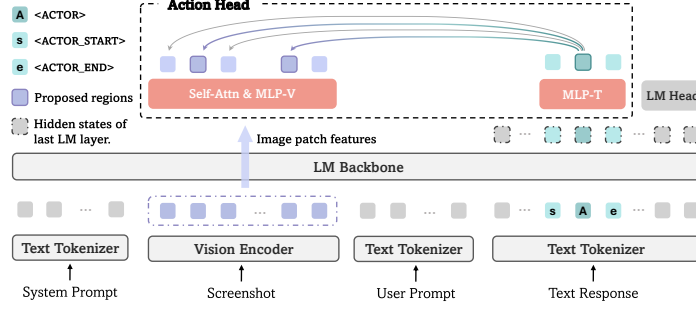
GUI 视觉定位。 给定一张 GUI 截图和一条自然语言指令, GUI 视觉定位旨在找到用于交互的目标区域。虽然这一任务在概念上与自然图像中的定位相关, 但 GUI 布局具有高语义密度和结构规律性, 因此带来独特挑战 [34, 8]。一种常见做法是把 GUI 定位表述为基于文本的坐标预测任务, 让模型把点位置 (*e.g.*, $x=\dots$, $y=\dots$) 作为语言 token 输出 [37, 38]。由于简单且兼容现有 LLM/VLM, 这种表述被广泛采用。为了提升性能, 已有工作同时扩展模型和训练数据 [39–43, 10, 44]。UGround [8] 提出一种用于合成多样 GUI 定位样本的数据流水线, 而 OS-Atlas [45] 提供了多平台数据集和统一的 GUI 动作模型。最近, Xu et al. [46] 提出了一种无需训练的方法, 通过利用 VLM 的内部注意力来执行 GUI 定位。

尽管这些方法取得了成功, 基于坐标的方法仍存在关键限制, 包括空间归纳偏置较弱、点监督存在歧义, 以及视觉特征与动作目标之间存在分辨率不匹配。本文提出了对主流坐标方法的一种有力替代方案: GUI-Actor, 一种面向 GUI 智能体的新型无坐标定位框架。它引入一个 `<ACTOR>` token, 并通过注意力式动作头直接关注相关图像 patch, 从而实现更接近人类的定位方式, 同时缓解基于坐标方法的局限。

3 GUI-Actor 的设计

考虑到基于文本的坐标生成存在空间-语义对齐较弱、监督目标有歧义等限制, 我们从人类与 GUI 交互的方式中获得启发。人类通常并不会计算精确坐标, 而是先从视觉上识别目标元素, 然后通过手指点击或移动鼠标光标直接作用于该元素。受此启发, GUI-Actor 探索了一种用于 GUI 视觉定位的新型架构: 我们首先引入一个特殊 token `<ACTOR>` 作为上下文锚点, 然后训练一个动作注意力头, 使该锚点关注与目标元素对应的图像 patch。最后, 我们提出一个定位验证器, 用于从注意力图导出的多个候选目标中选择语义上最合适的目标。

`<ACTOR>` Token 作为上下文锚点 给定一张截图图像 $\mathcal{I}$ 和一条指令 q , 基于坐标生成的方法通常训练 VLM 生成序列 $\{\mathbf{x}_{1:i-1}, \mathbf{x}_{i:i+m}, \mathbf{x}_{i+m+1:j-1}, \mathbf{x}_{j:j+n}, \mathbf{x}_{j+n+1:N}\}$, 其中 $m, n > 0$, N 是输出序列总长度, $i > 1$, 并且 $j - 1 > i + m + 1$ 。例如, 在 `pyautogui.click(x=0.123, y=0.234)` Xu et al. [7] 中, 片段 $\mathbf{x}_{i:i+m}$ 和 $\mathbf{x}_{j:j+n}$ 分别对应



(a) 注意力式动作头示意图。



(b) 图像 patch 标签

图 2: GUI-Actor 概览。(a) 动作头如何与 VLM 配合, 实现无坐标 GUI 定位。(b) 用于模型训练的空间感知多 patch 监督示意图。我们把所有被真实边界框部分或完全覆盖的图像 patch 标为正例 (1), 其余 patch 标为负例 (0)。

分词后的 x 坐标和 y 坐标。片段 $\mathbf{x}_{i+m+1:j-1}$ 表示二者之间的分隔符, 而其余部分捕获周围上下文。在本文中, 我们用三个特殊 token 替换坐标片段 $\{\mathbf{x}_{i:i+m}, \mathbf{x}_{i+m+1:j-1}, \mathbf{x}_{j:j+n}\}$, 以支持无坐标定位, 并更好地融合视觉输入和文本指令中的上下文:

$$\text{VLM}(\mathcal{I}, q) = \{\mathbf{x}_{1:i-1}, \langle \text{ACTOR_START} \rangle, \langle \text{ACTOR} \rangle, \langle \text{ACTOR_END} \rangle, \mathbf{x}_{i+3:N}\}. \quad (1)$$

我们使用 $\langle \text{ACTOR} \rangle$ 的最后一层隐藏状态, 即 $h_{\langle \text{ACTOR} \rangle}$, 作为上下文锚点, 用于在视觉 patch token 上计算动作注意力。

注意力式动作头 令 $v_1, \dots, v_M$ 表示 VLM 的视觉编码器从输入截图中提取的视觉 patch 特征, 其中每个 $v_i \in \mathbb{R}^d$ 。动作头从 $\langle \text{ACTOR} \rangle$ token 出发, 在这些视觉 patch 上计算注意力分布, 以识别目标动作区域。为了融入面向 GUI 的上下文信息, 我们首先对视觉 patch 特征应用自注意力层。这使模型能够在空间相关的 patch 之间聚合语义, 让属于同一个 GUI 元素的 patch 共享一致表示:

$$\tilde{v}_1, \dots, \tilde{v}_M = \text{SelfAttn}(v_1, \dots, v_M), \quad (2)$$

其中 $\tilde{v}_i \in \mathbb{R}^d$ 表示自注意力模块之后第 i 个 patch 的上下文文化特征。

接着，我们通过两个独立的 MLP，把 <ACTOR> token 表示 $h_{\text{<ACTOR>}}$ 和上下文化 patch 特征 $\tilde{v}_1, \dots, \tilde{v}_M$ 投影到共享嵌入空间，得到 z 和 $z_i \in \mathbb{R}^d$ ：

$$z = \text{MLP}_T(h_{\text{<ACTOR>}}), \quad z_i = \text{MLP}_V(\tilde{v}_i). \quad (3)$$

最后，我们计算 <ACTOR> token 与每个视觉 patch 之间的注意力分数。令 M 表示输入到 LM 骨干的图像特征总数，得到的注意力权重 $a_1, \dots, a_M$ 构成屏幕上的注意力图，指示与动作定位最相关的区域。

$$\alpha_i = \frac{z^\top z_i}{\sqrt{d}}, \quad a_i = \frac{\exp(\alpha_i)}{\sum_{j=1}^M \exp(\alpha_j)}, \quad \text{for } i = 1, \dots, M. \quad (4)$$

空间感知多 Patch 监督 我们方法的一个关键优势，是能够利用来自边界框监督的密集且具有空间结构的学习信号。不同于传统坐标方法依赖单个、可能有歧义的点击点，GUI-Actor 把所有被真实边界框部分或完全覆盖的图像 patch 作为正例 ($y_i=1$)，其余作为负例 ($y_i=0$)，其中 y_i 表示与 v_i 关联的标签。这使模型能够更有效地捕获可操作元素的完整空间范围。Figure 2b 给出了示意图。更多细节见附录 B。

我们使用下一 token 预测 (NTP) 损失和动作注意力损失的组合来训练模型：

$$\mathcal{L} = \mathcal{L}_{\text{NTP}} + \mathcal{L}_{\text{Action_Attn}}, \quad (5)$$

动作注意力损失定义为：

$$\mathcal{L}_{\text{Action_Attn}} = \sum_{i=1}^M p_i \log \frac{p_i}{a_i}, \quad p_i = \frac{y_i}{\sum_{j=1}^M y_j + \epsilon}, \quad i = 1, \dots, M, \quad (6)$$

其中 ϵ 是用于数值稳定性的小常数。

4 定位验证器

我们的注意力式动作定位模型有一个关键优势：它可以在单次前向传播中产生多个候选动作区域，而不增加额外推理成本。这是注意力式设计的自然结果，因为模型会同时为所有视觉 patch 分配分数。这种效率带来一个机会：我们无需只依赖得分最高的 patch，而可以引入一个轻量级验证步骤，从候选中选择语义上最合适的目标。

基于“验证通常比生成更容易”这一观察 [47]，我们提出一个 *Grounding Verifier*。它是一个轻量级 VLM 模块，以指令和带有视觉标记的截图作为输入，并预测被标记区域是否正确满足任务意图。该验证器作为决策细化层，使系统能够在执行前反思其动作。

数据与训练 验证器训练数据来自 OS-Atlas 数据集 [45]，该数据集覆盖桌面、移动端和 Web 领域。数据集提供形式为 (image, query, bounding box) 的三元组，其中每张图像与多条查询及其对应边界框相关联。对每个三元组，我们通过在边界框中心放置视觉

标记 (*i.e.* 空心红圈) 生成一个正例, 把它视为给定查询的正确定位点。为了构造负例, 我们采用两种策略: (1) 选择同一图像中错误边界框的中心; (2) 在目标区域外随机采样一个点。每个得到的训练样本格式化为元组 $(\mathcal{I}, \mathbf{x}, y)$, 其中 $\mathcal{I}$ 是带有标记点的图像, $\mathbf{x}$ 是对应语言指令, $y \in \{\text{'True'}, \text{'False'}\}$ 是真实标签, 表示该点是否正确满足指令。更多细节见附录 F.1。

我们从一个轻量级 VLM 微调验证器, 并使用标准监督学习。模型以 $(\mathcal{I}, \mathbf{x})$ 作为输入, 训练目标是生成正确 token y 。训练目标为交叉熵损失:

$$\mathcal{L}_{\text{Verifier}} = -\log P_{\theta_v}(y \mid \mathbf{I}, \mathbf{x}),$$

其中 P_{θ_v} 表示参数为 θ_v 的验证器模型输出概率。

推理 推理时, GUI-Actor 通过结合自然语言生成和视觉定位来预测最终动作位置。给定当前 GUI 状态和用户指令, GUI-Actor 首先通过标准解码生成智能体响应, 例如产生包含特殊 `<ACTOR>` token 的字符串 `pyautogui.click(<ACTOR_START><ACTOR><ACTOR_END>)`。随后, 我们提取 `<ACTOR>` 对应的隐藏状态, 并使用动作头在所有视觉 patch 上计算注意力。该注意力分布作为空间激活图, 用于识别执行预测动作时最相关的屏幕区域。

为了在 top- K 注意力加权 patch 中识别语义上最合适的区域, 我们用验证器 θ_v 为每个候选打分: 在图像 $\mathcal{I}$ 上标记该候选, 并评估其与指令 x 的对齐程度。对每个带标记图像, 验证器输出 `'True'` 和 `'False'` token 的概率, 我们将选择分数定义为:

$$s(\mathcal{I}, x) = \frac{P_{\theta_v}(\text{'True'} \mid \mathcal{I}, x)}{P_{\theta_v}(\text{'True'} \mid \mathcal{I}, x) + P_{\theta_v}(\text{'False'} \mid \mathcal{I}, x)}. \quad (7)$$

候选会按注意力权重降序评估; 一旦某个候选超过置信度阈值 (例如 $s(\mathcal{I}, x) > \gamma$), 我们便返回该候选, 不再继续评估。

5 实验

实现细节 我们使用 PyTorch 和 Huggingface Transformers 实现 GUI-Actor。除非另有说明, 我们采用 Qwen-2-VL-7B-Instruct [38] 作为 GUI-Actor 和基线的骨干 VLM, 以保证与已有最新方法公平比较。对于使用点监督 (*point sup.*) 和边界框监督 (*bbbox sup.*) 的 Aguvis-7B 基线复现, 我们直接使用 Aguvis [7] 提供的官方源代码。对动作头的所有配置, 我们使用与骨干 VLM 相同的维度。定位验证器从 UI-TARS-2B-SFT [10] 微调而来。推理时, 我们构造 $K = 20$ 个候选, 并在 ScreenSpot-Pro 上使用 $\gamma = 0.95$ 的置信度阈值, 在 ScreenSpot 和 ScreenSpot-v2 上使用 $\gamma = 0.8$ 。遵循 Aguvis [7], 我们把训练数据组织为 pyautogui 风格操作序列, 但按第 3 节所述, 用特殊 token 替换原始坐标。完整训练配方由多个公开 GUI 数据集构成, 包含约 $\sim 1\text{M}$ 张截图。GUI-Actor 和两个基线模型均使用表 7 总结的数据配方训练 1 个 epoch。更多数据集细节见 Appendix D。训练 GUI-Actor 时, 我们先冻结所有骨干 VLM 参数, 只训练动作头中新引入的组件

表 1: 在 *ScreenSpot-Pro* 上的性能比较。该基准包含更高分辨率界面和更大的领域偏移 (*e.g.*, 工业软件、多窗口布局), 可作为评估泛化能力的实用测试平台。

	开发	创意	CAD	科学	办公	OS	文本平均	图标平均	平均
GPT-4o	0.7	0.6	1.5	1.2	0.9	0	1.3	0	0.8
Claude Compute	12.6	16.8	11.9	25.8	26.9	8.1	23.4	7.1	17.1
OS-Atlas-4B	3.7	2.3	1.5	7.5	4.8	3.1	5.0	1.7	3.7
ShowUI-2B	9.4	5.3	1.9	10.6	13.5	6.6	10.8	2.6	7.7
UGround-V1-2B	27.4	26.7	14.6	34.3	38.3	17.9	-	-	26.6
UI-TARS-2B	26.4	27.6	14.6	39.8	42.6	14.3	39.6	8.4	27.7
GUI-Actor-2B	34.8	35.5	28.4	38.2	53.9	30.6	50.7	14.1	36.7
GUI-Actor-2B + Verifier	41.8	36.7	34.5	41.3	62.6	36.2	57.6	16.1	41.8
Qwen2-VL-7B	1.3	0.9	0.4	3.5	3.0	0.5	2.5	0.2	1.6
SeeClick-9.6B	0.3	0.6	1.9	2.0	0.9	1.5	1.8	0	1.1
Aria-UI-2-5.3B	8.4	14.7	6.1	18.1	16.1	2.6	17.1	2.0	11.3
OS-Atlas-7B	17.7	17.9	10.3	24.4	27.4	16.8	28.1	4.0	18.9
Aguvis-7B	16.1	21.4	13.8	34.6	34.3	19.4	-	-	22.9
UGround-V1-7B	28.1	31.7	14.6	39	49.6	24.5	-	-	31.1
UI-TARS-7B	36.1	32.8	18.0	50.0	53.5	24.5	47.8	16.2	35.7
GUI-Actor-7B	38.8	40.2	29.5	44.5	56.5	36.2	55.8	16.4	40.7
GUI-Actor-7B + Verifier	38.8	40.5	37.2	44.5	64.8	43.9	60.7	17.6	44.2
CogAgent-18B	8.0	5.6	6.1	13.4	10.0	3.1	12.0	0.8	7.7
UGround-72B-V1	31.1	35.8	13.8	50.0	51.3	25.5	-	-	34.5
UI-TARS-72B	40.8	39.6	17.2	45.7	54.8	30.1	50.9	17.5	38.1

(2B/7B 骨干分别约 $\sim 20\text{M}/\sim 100\text{M}$ 参数)。在这一热身阶段之后, 我们使用标准监督学习微调整个模型。

评测基准与指标 我们在三个成熟的 GUI 视觉定位基准上评测 GUI-Actor 和其他基线方法: ScreenSpot [34]、ScreenSpot-v2 [45] 和 ScreenSpot-Pro [48]。其中 ScreenSpot-Pro 包含更高分辨率界面和更大的领域偏移 (例如工业软件、多窗口布局), 是评估泛化能力的实用测试平台。评测指标采用 *Element Accuracy*, 衡量预测点击点落在目标元素真实边界框内的比例。更多基准信息见附录 E。

基线 我们主要将方法与相近规模 ($\sim 7\text{B}$ 参数) 的模型比较。基线包括 (i) GPT-4o [49]、Claude for Computer Use [50] 和 Gemini 2.0 [51] 等闭源模型, 以及 (ii) SeeClick [34]、ShowUI [39] 和 Magma [52] 等开源模型。我们特别强调若干与我们共享相同骨干的基线, 包括骨干 Qwen2-VL [38]、Aguvis-7B [7]、UGround-v1-7B [8] 和 UI-TARS-7B [10]。我们还比较 Qwen-2.5-VL 及其为骨干的模型, 例如 Jedi [44]。除非另有说明, 所有数值均来自原论文或 UI-TARS 基准 [10]。

主要结果 表 1、表 2、表 3 和表 4 展示了 ScreenSpot-Pro、ScreenSpot 和 ScreenSpot-v2 基准上的性能比较。我们的模型 *GUI-Actor-2B* 和 *GUI-Actor-7B* 在所有基准上都稳定超过已有最新方法, 其中 2B 模型甚至超过了许多 7B 竞争模型。唯一例外是 UI-TARS-7B 在某些设置中表现更强, 但它受益于显著更大的训练数据集, 其中包含公开数据和专有数据 (见 Figure 1)。此外, 它采用了更复杂的训练流程, 包括持续预训练、退火阶段以及最终的直接偏好优化 (DPO)。尽管我们的模型只使用监督微调, 在 ScreenSpot-Pro 上仍达到有竞争力甚至更优的结果, 展示出强能力和潜力。

表 2: 在 *ScreenSpot* 上的性能比较。

	移动端文本	移动端图标	桌面端文本	桌面端图标	Web 文本	Web 图标	平均
GPT-4	22.6	24.5	20.2	11.8	9.2	8.8	16.2
GPT-4o	20.2	24.9	21.1	23.6	12.2	7.8	18.3
Claude Computer Use	-	-	-	-	-	-	83.0
Gemini 2.0	-	-	-	-	-	-	84.0
UGround-V1-2B	89.4	72.0	88.7	65.7	81.3	68.9	77.7
UI-TARS-2B	93.0	75.5	90.7	68.6	84.3	74.8	82.3
GUI-Actor-2B	93.0	79.9	88.1	78.6	90.9	84.0	86.5
GUI-Actor-2B + Verifier	93.8	81.2	89.7	80.7	91.3	80.6	86.9
Qwen2-VL-7B	75.5	60.7	76.3	54.3	35.2	25.7	55.3
CogAgent-7B	67.0	24.0	74.2	20.0	70.4	28.6	47.4
SeeClick-9.6B	78.0	52.0	72.2	30.0	55.7	32.5	53.4
Magma-8B	60.4	58.5	75.3	52.9	69.1	52.0	60.3
Aguvis-G-7B	88.3	78.2	88.1	70.7	85.7	74.8	81.8
OS-Atlas-7B	93.0	72.9	91.8	62.9	90.9	74.3	82.5
Aguvis-7B	95.6	77.7	93.8	67.1	88.3	75.2	84.4
UGround-v1-7B	93.0	79.9	93.8	76.4	90.9	84.0	86.3
UI-TARS-7B	94.5	85.2	95.9	85.7	90.0	83.5	89.5
GUI-Actor-7B	94.9	82.1	91.8	80.0	91.3	85.4	88.3
GUI-Actor-7B + Verifier	96.0	83.0	93.8	82.1	92.2	87.4	89.7
UI-TARS-72B	94.9	82.5	89.7	88.6	88.7	85.0	88.4
Aguvis-72B	94.5	85.2	95.4	77.9	91.3	85.9	89.2
UGround-V1-72B	94.1	83.4	94.9	85.7	90.4	87.9	89.4

表 3: 在 *ScreenSpot-v2* 上的性能比较。[†] 表示我们在 Huggingface 官方模型上自行评测得到的结果。

	移动端文本	移动端图标	桌面端文本	桌面端图标	Web 文本	Web 图标	平均
Operator	47.3	41.5	90.2	80.3	92.8	84.3	70.5
GPT-4o + OmniParser-v2	95.5	74.6	92.3	60.9	88.0	59.6	80.7
OS-Atlas-4B	87.2	59.7	72.7	46.4	85.9	63.1	71.9
UI-TARS-2B	95.2	79.1	90.7	68.6	87.2	78.3	84.7
GUI-Actor-2B	95.0	82.2	92.2	81.8	92.9	82.7	88.6
GUI-Actor-2B + Verifier	95.9	81.5	94.3	82.9	93.6	82.8	89.3
SeeClick-9.6B	78.4	50.7	70.1	29.3	55.2	32.5	55.1
Magma-8B	62.8	53.4	80.0	57.9	67.5	47.3	61.5
OS-Atlas-7B	95.2	75.8	90.7	63.6	90.6	77.3	84.1
Aguvis-7B [†]	95.5	77.3	95.4	77.9	91.0	72.4	86.0
UGround-V1-7B [†]	95.0	83.3	95.0	77.8	92.1	77.2	87.6
UI-TARS-7B	96.9	89.1	95.4	85.0	93.6	85.2	91.6
GUI-Actor-7B	96.5	84.3	91.7	84.1	93.9	82.3	89.5
GUI-Actor-7B + Verifier	97.2	84.8	94.3	85.0	94.0	85.2	90.9
UI-TARS-72B	94.8	86.3	91.2	87.9	91.5	87.7	90.3

稳健的分布外泛化 如表 1 所示, *GUI-Actor-2B* 和 *GUI-Actor-7B* 在 *ScreenSpot-Pro* 上都表现强劲。该基准是一个分布外基准, 特点是分辨率更高, 并且与训练数据存在显著领域偏移。我们的 2B 和 7B 模型分别超过此前最新 UI-TARS 模型 +9.0 和 +5.0 分。我们将这一提升归因于显式空间-语义对齐: 不同于 UI-TARS 等基于坐标的方法, GUI-Actor 使用注意力式动作头, 把语义线索直接定位到离散视觉区域。这一设计引入了更强的空间归纳偏置, 并天然匹配现代视觉骨干的 patch 表示。因此, GUI-Actor 更适合对局部视觉内容进行推理, 从而在多样屏幕分辨率和 UI 布局中实现稳健泛化。3(c) 中也能看到这种鲁棒性的进一步证据: 随着训练进行, *GUI-Actor-2B* 和 *GUI-Actor-7B* 在域外 *ScreenSpot-Pro* 基准上没有持续过拟合; 其准确率从早期下降中恢复, 随后逐渐上升并稳定。相比之下, 基线性能在早期达到峰值后持续下降。

表 4: 基于 Qwen-2.5-VL 骨干的模型性能比较。

<i>ScreenSpot-Pro:</i>	开发	创意	CAD	科学	办公	OS	平均
Qwen2.5-VL-3B	21.4	25.8	18.4	29.5	40.9	20.4	25.9
Qwen2.5-VL-7B	29.1	24.9	13.8	31.1	45.7	22.4	27.6
Jedi-3B	38.1	34.6	23.0	38.6	57.0	25.0	36.1
Jedi-7B	27.4	34.0	32.2	52.4	68.7	26.0	39.5
GUI-Actor-3B	39.8	36.7	34.1	49.6	61.3	35.2	42.2
GUI-Actor-3B + Verifier	43.8	37.8	43.3	48.4	63.5	42.9	45.9
GUI-Actor-7B	38.1	41.4	38.3	50.8	63.0	38.8	44.6
GUI-Actor-7B + Verifier	43.1	41.9	48.3	47.2	65.7	43.4	47.7
<i>ScreenSpot-v2:</i>	移动端文本	移动端图标	桌面端文本	桌面端图标	Web 文本	Web 图标	平均
Qwen2.5-VL-3B	93.4	73.5	88.1	58.6	88.0	71.4	80.9
Qwen2.5-VL-7B	97.6	87.2	90.2	74.2	93.2	81.3	88.8
Jedi-3B	96.6	81.5	96.9	78.6	88.5	83.7	88.6
Jedi-7B	96.9	87.2	95.9	87.9	94.4	84.2	91.7
GUI-Actor-3B	97.6	83.4	96.9	83.6	94.0	85.7	91.0
GUI-Actor-3B + Verifier	98.3	85.3	96.9	87.9	95.3	86.7	92.4
GUI-Actor-7B	97.6	88.2	96.9	85.7	93.2	86.7	92.1
GUI-Actor-7B + Verifier	96.9	89.6	97.4	86.4	95.7	84.7	92.5

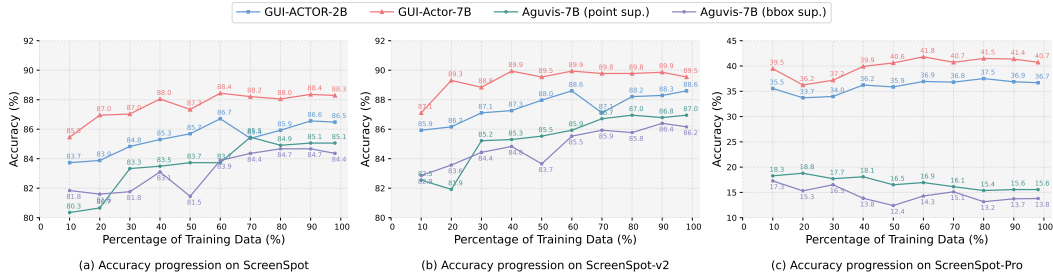


图 3: 训练步数上的准确率变化。

提升样本效率 图 3 展示了 GUI-Actor 的设计如何相较于基于坐标的基线带来更高样本效率。GUI-Actor 只使用约 $\sim 60\%$ 的训练数据，就在 ScreenSpot 和 ScreenSpot-v2 上达到最终准确率，超过了在 80-90% 数据后才趋于平台期的 AGUAVIS 点监督和框监督模型。这种效率来自 GUI-Actor 通过动作头实现的显式空间-语义对齐，使其能够直接在视觉骨干的原生 patch 分辨率上定位，从而避免阻碍基线方法的粒度不匹配。此外，我们的多 patch 监督策略可以自然处理坐标生成方法中的监督歧义，提供密集且有空间结构的监督信号。

在不牺牲通用能力的情况下赋予骨干 VLM GUI 定位能力。 我们引入一个变体 *GUI-Actor-LiteTrain*，其中冻结所有骨干 VLM 参数，只训练新引入的动作头组件和特殊 token。该设置探索 GUI-Actor 如何在不削弱 VLM 通用能力的情况下赋予其 GUI 定位能力。如表 5 所示，*GUI-Actor-LiteTrain* 相比未修改的骨干 VLM 带来显著性能提升。在定位验证器帮助下，它甚至可以接近完全微调的坐标生成模型。这些结果说明，骨干 VLM 已经具备较强的 UI 截图感知理解能力。因此，训练模型以文本形式生成坐标，可能主要聚焦于坐标映射，而对 UI 元素语义理解的贡献有限。更重要的是，*GUI-Actor-LiteTrain* 保留了骨干原有语言和视觉语言能力，说明轻量级集成可以在不损害通用性的情况下启用定位。

表 5: 轻量训练 (-*LiteTrain*) 分析: 冻结骨干 VLM (*i.e.*, Qwen-2-VL), 训练时只更新动作头和特殊 token 中新引入的参数。

方法	更新参数量	ScreenSpot-Pro	ScreenSpot	ScreenSpot-v2
GUI-Actor-2B-LiteTrain	19M	25.4	71.4	73.9
GUI-Actor-2B-LiteTrain + Verifier	19M	34.0	79.2	82.3
GUI-Actor-7B-LiteTrain	103M	22.9	73.5	74.9
GUI-Actor-7B-LiteTrain + Verifier	103M	35.8	81.3	83.8

通过定位验证器提升性能 表 1、表 2、表 3 和表 5 的结果表明, 定位验证器稳定提升性能, 说明它能有效增强定位准确率。在 ScreenSpot-Pro 上提升尤其明显, 它将 GUI-Actor-7B 提升近 4 分, 并将 GUI-Actor-7B-LiteTrain 提升了令人瞩目的 13 分。此外, 我们在附录 G.1 中研究了 Verifier Self-Aggregation 策略的收益, 并在附录 G.2 中评估验证器对其他基线模型的适用性。结果表明, 我们的验证器非常稳健, 并且特别适合 GUI-Actor, 因为它只需单次前向传播即可生成多样区域候选。

消融研究 表 6 展示了消融研究结果, 其中“bbox sup.”和“point sup.”分别表示训练模型以自然语言格式预测真实边界框坐标或动作点坐标。结果表明, 使用坐标生成训练的模型 (无论是边界框监督还是点监督) 在各个基准上都稳定弱于 *GUI-Actor-7B*, 突出了我们提出的动作头所实现的显式空间-语义对齐的有效性和必要性。有趣的是, 尽管 *Aguvis-7B (bbox sup.)* 拥有更多空间信息, 它的表现与 *Aguvis-7B (point sup.)* 相近甚至更差。这说明, 如果缺少架构机制或空间归纳偏置, 这些基于坐标生成的方法仍会与底层视觉表示脱节, 从而限制其泛化和定位能力。

表 6: 在 *ScreenSpot-Pro*、*ScreenSpot* 和 *ScreenSpot-v2* 上的消融研究。

<i>ScreenSpot-Pro</i> :	开发	创意	CAD	科学	办公	OS	平均
GUI-Actor-7B	38.8	40.2	29.5	44.5	56.5	36.2	40.7
Aguvis-7B (bbox sup.)	12.4	17.0	1.5	18.1	21.7	11.7	13.8
Aguvis-7B (point sup.)	15.7	19.4	3.8	17.3	24.4	11.7	15.6
<i>ScreenSpot</i> :	移动端文本	移动端图标	桌面端文本	桌面端图标	Web 文本	Web 图标	平均
GUI-Actor-7B	94.9	82.1	91.8	80.0	91.3	85.4	88.3
Aguvis-7B (bbox sup.)	92.3	73.4	92.3	76.4	92.6	74.8	84.4
Aguvis-7B (point sup.)	92.3	79.0	93.3	71.4	92.6	75.2	85.1
<i>ScreenSpot-v2</i> :	移动端文本	移动端图标	桌面端文本	桌面端图标	Web 文本	Web 图标	平均
GUI-Actor-7B	96.5	84.3	91.7	84.1	93.9	82.3	89.5
Aguvis-7B (bbox sup.)	92.3	73.4	92.3	76.4	92.6	74.8	84.4
Aguvis-7B (point sup.)	96.1	80.1	96.1	74.6	93.6	74.3	87.0

无额外推理成本的多区域预测 由于采用注意力式定位机制, GUI-Actor 可以在单次前向传播中生成多个候选动作区域, 而不产生额外推理成本。为了评估这些高概率区域的有效性, 我们使用 Hit@k 指标, 其中 k 表示考虑的 top 排名预测数量。图 4a 显示, GUI-Actor 从 Hit@1 到 Hit@3 有显著提升, 而基线的差距相对较小。在分析中我们观察到, 对于基于坐标生成的基线, 即使进行多次采样, 输出也大多相同, 例如从 (0.898, 0.667) 轻微变化到 (0.899, 0.666)。相比之下, 我们的模型在单次前向传播中从注意力分布同时产生多个候选区域。这些候选互斥, 天然促进多样性, 并提高捕获所有有效动

作区域的机会。图 4b 给出了一个定性示例，展示我们的方法能够成功识别动作执行所需的所有真实区域。

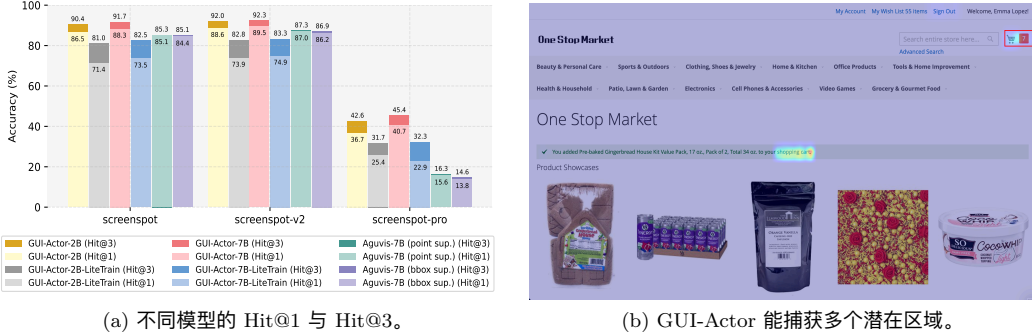


图 4: (a) 不同方法的 Hit@1 和 Hit@3。对于 Aguviz 基线，我们用 $\text{temperature} = 1.0$ 、 $\text{top_p} = 0.95$ 运行 3 次推理。(b) 多区域预测示意。在该示例中，指令为“check shopping cart”，中央的“shopping cart”文本可点击，而真实标注只有右上角图标。

OS-World-W 上的在线评测 为了评估所提出系统在真实世界中的有效性，我们在 OS-World-W 上进行了在线评测。OS-World-W 是 OS-World 基准的一个精选子集，包含 49 个 Windows 专用任务，涉及办公和多应用场景中的复杂多步交互。我们使用 GPT-4o 作为规划器，并使用 GUI-Actor-7B 进行动作定位。与领先的视觉定位基线 Aguviz-7B、NAVI [53] 和 OmniAgent [54] 相比，*GUI-Actor-7B* 展现出有潜力的性能，任务成功率达到 12.2%，超过 OmniAgent 和 NAVI (均为 10.2%)，并显著超过 *Aguvis-7B (point sup.)* (4.0%)。更多细节见附录 H。

6 结论

我们提出了 GUI-Actor，一种面向 GUI 智能体的新型无坐标视觉定位框架。不同于主流基于文本的坐标生成范式，GUI-Actor 引入专用的 `<ACTOR>` token，通过关注目标视觉 patch，直接在屏幕上定位 GUI 元素。该机制显式对齐空间视觉特征与指令中的语义信号，并天然支持基于边界框的多 patch 监督，从而缓解单点预测固有的歧义。得益于单次前向传播中提出多个候选区域的能力，GUI-Actor 进一步使用轻量级验证器在推理时选择最可信目标。多个基准上的实验表明，GUI-Actor 优于最新方法，并且对未见过的布局和屏幕分辨率具有更强泛化能力。我们进行了大量分析以理解框架中各组件的有效性，凸显其推动视觉 GUI 智能体发展的潜力。

致谢

我们感谢 Boyu Gou 提供边界框训练数据，并感谢 Yiheng Xu、Qiushi Sun、Zichen Ding 和 Fangzhi Xu 的宝贵讨论与深刻建议。

References

- [1] Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, et al. Ufo: A ui-focused agent for windows os interaction. *arXiv preprint arXiv:2402.07939*, 2024.
- [2] Chaoyun Zhang, He Huang, Chiming Ni, Jian Mu, Si Qin, Shilin He, Lu Wang, Fangkai Yang, Pu Zhao, Chao Du, et al. Ufo2: The desktop agents. *arXiv preprint arXiv:2504.14603*, 2025.
- [3] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- [4] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v (ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614*, 2024.
- [5] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, et al. Large language model-brained gui agents: A survey. *arXiv preprint arXiv:2411.18279*, 2024.
- [6] Lu Wang, Fangkai Yang, Chaoyun Zhang, Junting Lu, Jiaxu Qian, Shilin He, Pu Zhao, Bo Qiao, Ray Huang, Si Qin, et al. Large action models: From inception to implementation. *arXiv preprint arXiv:2412.10047*, 2024.
- [7] Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous gui interaction. *arXiv preprint arXiv:2412.04454*, 2024.
- [8] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents. *arXiv preprint arXiv:2410.05243*, 2024.
- [9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=YicbFdNTTy>.
- [10] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.
- [11] Yuxuan Sun, Chong Sun, Dong Wang, You He, and Huchuan Lu. Roi pooled correlation filters for visual tracking. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5783–5791, 2019.
- [12] Xizhou Zhu, Dazhi Cheng, Zheng Zhang, Stephen Lin, and Jifeng Dai. An empirical study of spatial attention mechanisms in deep networks. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6688–6697, 2019.
- [13] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A

- dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.
- [14] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
 - [15] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114, 2023.
 - [16] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.
 - [17] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*, 2024.
 - [18] Chengyou Jia, Minnan Luo, Zhuohang Dang, Qiushi Sun, Fangzhi Xu, Junlin Hu, Tianbao Xie, and Zhiyong Wu. Agentstore: Scalable integration of heterogeneous agents as specialized generalist computer assistant. *arXiv preprint arXiv:2410.18603*, 2024.
 - [19] Qiushi Sun, Zhirui Chen, Fangzhi Xu, Kanzhi Cheng, Chang Ma, Zhangyue Yin, Jianing Wang, Chengcheng Han, Renyu Zhu, Shuai Yuan, et al. A survey of neural code intelligence: Paradigms, advances and beyond. *arXiv preprint arXiv:2403.14734*, 2024.
 - [20] Qiushi Sun, Zhoumianze Liu, Chang Ma, Zichen Ding, Fangzhi Xu, Zhangyue Yin, Haiteng Zhao, Zhenyu Wu, Kanzhi Cheng, Zhaoyang Liu, et al. Scienceboard: Evaluating multimodal autonomous agents in realistic scientific workflows. *arXiv preprint arXiv:2505.19897*, 2025.
 - [21] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*, pages 1–20, 2025.
 - [22] Chaoyun Zhang, Shilin He, Liqun Li, Si Qin, Yu Kang, Qingwei Lin, and Dongmei Zhang. Api agents vs. gui agents: Divergence and convergence. *arXiv preprint arXiv:2503.11069*, 2025.
 - [23] Wentong Chen, Junbo Cui, Jinyi Hu, Yujia Qin, Junjie Fang, Yue Zhao, Chongyi Wang, Jun Liu, Guirong Chen, Yupeng Huo, et al. Guicourse: From general vision language models to versatile gui agents. *arXiv preprint arXiv:2406.11317*, 2024.
 - [24] Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Dingyu Zhang, Peng Gao, Shuai Ren, and Hongsheng Li. Amex: Android multi-annotation expo dataset for mobile gui agents. *arXiv preprint arXiv:2407.17490*, 2024.
 - [25] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. *arXiv preprint arXiv:2406.08451*, 2024.

- [26] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. Ferret-ui: Grounded mobile ui understanding with multimodal llms. In *European Conference on Computer Vision*, pages 240–255. Springer, 2024.
- [27] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Androidinthewild: A large-scale dataset for android device control. *Advances in Neural Information Processing Systems*, 36:59708–59728, 2023.
- [28] Jiwen Zhang, Jihao Wu, Yihua Teng, Minghui Liao, Nuo Xu, Xiao Xiao, Zhongyu Wei, and Duyu Tang. Android in the zoo: Chain-of-action-thought for gui agents. *arXiv preprint arXiv:2403.02713*, 2024.
- [29] Jiani Zheng, Lu Wang, Fangkai Yang, Chaoyun Zhang, Lingrui Mei, Wenjie Yin, Qingwei Lin, Dongmei Zhang, Saravan Rajmohan, and Qi Zhang. Vem: Environment-free exploration for training gui agent with value environment model. *arXiv preprint arXiv:2502.18906*, 2025.
- [30] Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. *arXiv preprint arXiv:2412.19723*, 2024.
- [31] Kanzhi Cheng, Yantao Li, Fangzhi Xu, Jianbing Zhang, Hao Zhou, and Yang Liu. Vision-language models can self-improve reasoning via reflection. *arXiv preprint arXiv:2411.00855*, 2024.
- [32] Junlei Zhang, Zichen Ding, Chang Ma, Zijie Chen, Qiushi Sun, Zhenzhong Lan, and Junxian He. Breaking the data barrier—building gui agents through task generalization. *arXiv preprint arXiv:2504.10127*, 2025.
- [33] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.
- [34] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. Seeclick: Harnessing gui grounding for advanced visual gui agents. *arXiv preprint arXiv:2401.10935*, 2024.
- [35] OpenAI. Computer-using agent. Available at: <https://openai.com/index/computer-using-agent>, 2025.
- [36] Anthropic. Claude computer use. Available at: <https://www.anthropic.com/news/developing-computer-use>, 2024.
- [37] Keqin Chen, Zhao Zhang, Weili Zeng, Richong Zhang, Feng Zhu, and Rui Zhao. Shikra: Unleashing multimodal llm’s referential dialogue magic. *arXiv preprint arXiv:2306.15195*, 2023.
- [38] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- [39] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Weixian Lei, Lijuan Wang, and Mike Zheng Shou. Showui: One vision-language-action model for gui visual agent. *arXiv preprint arXiv:2411.17465*, 2024.

- [40] Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-ui: Visual grounding for gui instructions. *arXiv preprint arXiv:2412.16256*, 2024.
- [41] Pawel Pawlowski, Krystian Zawistowski, Wojciech Lapacz, Marcin Skorupa, Adam Wiacek, Sebastien Postansque, and Jakub Hoscilowicz. Tyneclick: Single-turn agent for empowering gui automation. *arXiv preprint arXiv:2410.11871*, 2024.
- [42] Fei Tang, Yongliang Shen, Hang Zhang, Siqi Chen, Guiyang Hou, Wenqi Zhang, Wenqiao Zhang, Kaitao Song, Weiming Lu, and Yueting Zhuang. Think twice, click once: Enhancing gui grounding via fast and slow systems. *arXiv preprint arXiv:2503.06470*, 2025.
- [43] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [44] Tianbao Xie, Jiaqi Deng, Xiaochuan Li, Junlin Yang, Haoyuan Wu, Jixuan Chen, Wenjing Hu, Xinyuan Wang, Yuhui Xu, Zekun Wang, Yiheng Xu, Junli Wang, Doyen Sahoo, Tao Yu, and Caiming Xiong. Scaling computer-use grounding via user interface decomposition and synthesis, 2025.
- [45] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. *arXiv preprint arXiv:2410.23218*, 2024.
- [46] Hai-Ming Xu, Qi Chen, Lei Wang, and Lingqiao Liu. Attention-driven gui grounding: Leveraging pretrained multimodal large language models without fine-tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8851–8859, 2025.
- [47] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [48] Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. Screenspot-pro: Gui grounding for professional high-resolution computer use. *arXiv preprint arXiv:2504.07981*, 2025.
- [49] OpenAI. Introducing gpt-4o. Available at: <https://openai.com/index/hello-gpt-4o>, 2024.
- [50] Siyuan Hu, Mingyu Ouyang, Difei Gao, and Mike Zheng Shou. The dawn of gui agent: A preliminary case study with claude 3.5 computer use, 2024. URL <https://arxiv.org/abs/2411.10323>.
- [51] Google. Introducing gemini 2.0. Available at: <https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024>, 2024.
- [52] Jianwei Yang, Reuben Tan, Qianhui Wu, Ruijie Zheng, Baolin Peng, Yongyuan Liang, Yu Gu, Mu Cai, Seonghyeon Ye, Joel Jang, et al. Magma: A foundation model for multimodal ai agents. *arXiv preprint arXiv:2502.13130*, 2025.
- [53] Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, et al. Windows agent arena: Evaluating multi-modal os agents at scale. *arXiv preprint arXiv:2409.08264*, 2024.

- [54] Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203*, 2024.
- [55] Wei Li, William E Bishop, Alice Li, Christopher Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems*, 37:92130–92154, 2024.
- [56] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin A Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.
- [57] Chongyang Bai, Xiaoxue Zang, Ying Xu, Srinivas Sunkara, Abhinav Rastogi, Jindong Chen, et al. Uibert: Learning generic multimodal representations for ui understanding. *arXiv preprint arXiv:2107.13731*, 2021.
- [58] Yang Li, Jiacong He, Xin Zhou, Yuan Zhang, and Jason Baldridge. Mapping natural language instructions to mobile ui action sequences. *arXiv preprint arXiv:2005.03776*, 2020.
- [59] Yang Li, Gang Li, Luheng He, Jingjie Zheng, Hong Li, and Zhiwei Guan. Widget captioning: Generating natural language description for mobile user interface elements. *arXiv preprint arXiv:2010.04295*, 2020.

A 局限性

我们的注意力式动作生成特别适合 GUI 环境，因为图标、文本行等界面元素通常呈现规则几何模式，并天然适配基于 patch 的检测。然而，一个限制来自我们采用的基础模型：骨干 VLM (*e.g.* Qwen2-VL) 使用固定 patch 大小为 28×28 像素的 Naive Dynamic Resolution 策略。当处理非常小的界面元素时（例如小于 10×10 像素的图标），这会带来挑战，因为这类细粒度细节可能无法被充分表示。尽管这一挑战并非我们方法独有，但在需要高精度控制的任务中会更加明显，例如 CAD 工具等专业软件。虽然本文引入了一个简单缓解策略，但要完全解决该限制，未来可能需要更实质性的进展，例如提升视觉编码器的感知分辨率，或引入基于偏移量的空间细化。

B 多 Patch 监督细节

我们方法的一个关键优势在于，能够利用来自边界框的密集且有空间结构的监督。不同于依赖单个、可能有歧义点击点的传统坐标方法，GUI-Actor 能够在整个目标区域上进行监督，从而更有效地捕获可操作元素的空间范围。Figure 2b 给出了一个示例。

具体而言，我们把每个真实边界框转换为 $W \times H$ 图像 patch 网格上的二值掩码。给定归一化边界框 $b = [\text{left}, \text{top}, \text{right}, \text{bottom}] \in [0, 1]^4$ ，我们将归一化坐标缩放到 patch 网格分辨率：

$$([\text{left} \cdot W], [\text{top} \cdot H], [\text{right} \cdot W], [\text{bottom} \cdot H]). \quad (8)$$

这种取整保证所有被边界框部分或完全覆盖的 patch 都纳入监督区域。索引位于所得网格对齐区域内的所有 patch 标为正例（即掩码值为 1），其他 patch 的值设为 0。由此得到一个与图像 patch 序列对齐的二值向量 $\mathbf{y} \in \{0, 1\}^M$ ，其中 $M = W \times H$ 。

我们将动作头损失定义为预测注意力分布 $\{a_1, \dots, a_M\}$ 与由二值掩码 $\mathbf{y} \in \{0, 1\}^M$ 导出的归一化目标分布 $\mathbf{p}$ 之间的 KL 散度：

$$p_i = \frac{y_i}{\sum_{j=1}^M y_j + \epsilon}, \text{ for } i = 1, \dots, M; \quad \mathcal{L}_{\langle \text{ACTOR} \rangle} = \sum_{i=1}^M p_i \log \frac{p_i}{a_i}, \quad (9)$$

其中 ϵ 是用于数值稳定性的小常数。

C GUI-Actor 注意力图可视化

为了更好地理解 GUI-Actor 的定位行为，我们在图 5 中提供了额外示例，将其注意力图叠加到原始输入图像上。

下面的 Python 代码概述了可视化过程：从原始注意力分数出发，我们先对其归一化并重塑为图像维度，再应用 colormap 以便解释，最后把注意力热力图与原图融合。这会生成一种直观叠加图，突出模型决策时关注的区域。

```

1 width, height = image.size
2 W, H = attention_map_size
3 # This may differ from width // patch_size due to image reshaping.
4 # Qwen2-VL's Naive Dynamic Resolution may require W // 2 and H // 2.
5
6 scores = np.array(json_prediction['attn_scores'][0]).reshape(H, W)
7
8 # Normalize the attention weights for coherent visualization
9 scores_norm = (scores - scores.min()) / (scores.max() - scores.min())
10
11 # Resize the attention map to match the image size. You can choose different resize
    strategies, such as NEAREST and BILINEAR.
12 score_map = Image.fromarray((scores_norm * 255).astype(np.uint8)).resize((width, height),
    resample=Image.BILINEAR)
13
14 # Apply colormap
15 colormap = plt.get_cmap('jet')
16 colored_score_map = colormap(np.array(score_map) / 255.0) # returns RGBA
17 colored_score_map = (colored_score_map[:, :, :3] * 255).astype(np.uint8)
18 colored_overlay = Image.fromarray(colored_score_map)
19
20 # Blend with original image
21 blended = Image.blend(image, colored_overlay, alpha=0.3)

```

Listing 1: 在图像上叠加注意力分数图的 Python 代码。

D GUI-Actor 使用的训练数据集

我们从若干公开可用的高质量 GUI 数据集中构建训练数据。Table 7 给出了汇总统计。注意，我们排除了 Wave-UI 中与下游任务测试集重叠的样本。

表 7: GUI-Actor 使用的训练数据集概览。

数据集	元素数量	截图数量	平台
Uground Web-Hybrid [8]	8M	775K	Web
GUI-Env [23]	262K	70K	Web
GUI-Act [23]	42K	13K	Web
AndroidControl [55]	47K	47K	Android
AMEX [24]	1.2M	100K	Android
Wave-UI ¹	50K	7K	Hybrid
总计	9.6M	1M	-

E GUI 视觉定位基准

在这些基准中，每张截图都配有由人工标注者撰写的自然语言指令，通常描述目标元素的内容或功能，例如“the new project button”或“switch to weekly view in the calendar”。智能体需要根据给定指令识别屏幕上对应元素的位置。**ScreenSpot** 是第一个专门为 GUI 视觉定位设计的基准，包含 1,272 条单步指令及其对应目标元素。它覆盖广泛的 GUI 平台，包括移动端（Android 和 iOS）、桌面端（macOS 和 Windows）以及 Web 环境，并将元素分为基于文本的元素和图标元素。**ScreenSpot-v2** 是 ScreenSpot 的修正版，修复了标注错误和有歧义的指令，同时保持样本总数不变。

ScreenSpot-Pro 是近期提出的挑战性基准，面向高分辨率专业场景。它包含 1,581 个任务，由专家在跨越三个操作系统的 23 个专业应用中标注。相比 ScreenSpot，ScreenSpot-Pro 具有更高分辨率截图，并且与大多数定位预训练数据存在更大的领域差异，例如工业软件和多窗口界面。我们将其性能视为 GUI 视觉定位模型泛化能力的实际估计。

F 定位验证器的更多细节

F.1 数据构造

我们从 OS-Atlas 数据集 [45] 构造验证器训练集，该数据集覆盖桌面端、移动端和 Web 领域。原始数据由 (image, query, bounding box) 形式的三元组组成，其中每张图像配有多条查询及其对应边界框。对每个三元组，我们通过边界框中心放置标记来生成一个**正例**，并将其视为给定查询的正确定位点。为了创建**负例**，我们采用两种策略：(1) 选择同一图像中另一个边界框的中心，以模拟语义上合理但错误的位置；(2) 在正确边界框外随机采样一个点，以模拟无关动作。如图 6 所示，每个候选点都在图像上用空心

¹<https://huggingface.co/datasets/agentsea/wave-ui>

红圈标记。该过程为每个查询产生两个带标签样本：一个正例和一个负例，格式为：

{image with correct point, query, `True'}, {image with wrong point, query, `False'}.

总而言之，我们构造了一个包含 730K 个样本的平衡训练集，正例和负例数量相同。表 8 列出了数据集概览。

表 8: 用于训练 Grounding Verifier 的数据集概览，包含正例和负例。由于可以从单张截图生成多个正例和负例样本，我们的数据集规模可能超过原始数据集规模。

数据集	截图数量	平台
SeeClick [34]	147K	Web
FineWeb [56]	123K	Web
UIbert [57]	17K	Mobile
AMEX [24]	155K	Android
RICOSCA [58]	30K	Android
Widget Captioning [59]	22K	Mobile
Linux-desktop [45]	9K	Linux
Windows-desktop [45]	220K	Windows
MacOS-desktop [45]	7K	MacOS
总计	730K	-

F.2 Patch 选择

给定注意力图中的 top M 候选 patch，我们的目标是选择图像中最符合用户指令的一个位置。一个直接做法是在每个 patch 中心绘制标记，并用验证器评估该位置满足指令 x 的程度。具体而言，对于每张带标记点的候选图像 I ，我们用验证器 θ_v 计算预测 `True' 或 `False' token 的概率： $P_{\text{true}} = P_{\theta_v}(\text{'True'}|I, x)$ 和 $P_{\text{false}} = P_{\theta_v}(\text{'False'}|I, x)$ 。随后，我们把每个位置的分数定义为 `True' token 的归一化概率： $s(I, x) = \frac{P_{\text{true}}}{P_{\text{true}} + P_{\text{false}}}$ 。

该方法的一个关键限制是，每个 patch（通常为 28×28 像素）可能错过位于两个相邻 patch 之间的小图标，从而导致目标定位不准确。为了解决这一点，我们引入一个简单而有效的细化策略。我们首先对 4 连通的相邻 patch 聚类，并基于各 patch 的验证器分数计算加权中心。这样可以生成位于相邻 patch 之间的候选点，并在不直接修改基础模型 patch 大小的情况下提高定位精度。

在实现中，我们最多使用 $M = 20$ 个最高分 patch，并过滤掉注意力权重低于最大注意力权重 20% 的 patch。随后，我们对相邻 patch 聚类，计算这些簇的加权中心，并将其加入 M 个候选位置集合。每个候选位置都用 $s(I, x)$ 打分，并选择得分最高者。给定候选坐标 (x, y) ，我们以 (x, y) 为中心裁剪大小为 $l_{\text{crop}} \times l_{\text{crop}}$ 的方形图像区域。实现如下：

```

1 image.crop((
2     max(0, x - l_crop//2),
3     max(0, y - l_crop//2),
4     min(image.size[0], x + l_crop//2),
5     min(image.size[1], y + l_crop//2)
6 ))

```

表 9: ScreenSpot-Pro 上的 Verifier Self-aggregation。

	开发	创意	CAD	科学	办公	OS	文本平均	图标平均	平均
GUI-Actor-7B + Verifier	40.1	39.0	37.2	47.2	63.5	41.8	61.1	16.7	44.2
GUI-Actor-7B + Verifier Self-aggregation	39.5	40.2	38.3	44.9	63.9	44.9	61.3	17.4	44.5

所有任务中我们都设置 $l_{\text{crop}} = 1000$ 像素。为了降低计算成本，如果某个候选位置达到高置信度分数（例如 $s(I, x) > 0.95$ ），我们会立即返回该位置，而不再评估剩余候选。在实验中，我们对 ScreenSpot-Pro 任务设置阈值 0.95，对 ScreenSpot 和 ScreenSpot-v2 设置 0.8。较低阈值会减少对验证器的依赖，更多信任定位模型输出，适用于定位模型本身已经很准确的情况。相反，较高阈值会促使验证器更仔细地评估每个候选，这在 ScreenSpot-Pro 等定位模型较不可靠的场景中更有益。

G 使用验证器改进定位

G.1 通过验证器自聚合增强生成

本节探索如何通过一种简单而有效的技术进一步利用验证器能力，我们称之为 Verifier Self-Aggregation (VS)。其思想是在多个尺度上裁剪输入图像，并计算每个裁剪图的验证器分数，然后对这些分数取平均，得到更稳健的最终预测。该方法在捕获细粒度局部信息（较小裁剪）和保留更宽上下文（较大裁剪）之间取得平衡。具体而言，我们在 ScreenSpot-Pro 实验中使用两个裁剪尺寸： $l_{\text{crop}} = 1200$ 和 $l_{\text{crop}} = 1400$ 。表 9 的结果表明，验证器自聚合可以提升 ScreenSpot-Pro 上的性能。验证器自聚合提供了一种简单而有效的策略来增强验证器鲁棒性，同时也说明未来需要更稳健的验证器。

G.2 与使用验证器的基线比较

为了进一步验证定位验证器的有效性，我们把它接入 AGUVIS 基线：在推理时采样多个候选位置，并选择验证器分数最高的位置。具体而言，AGUVIS 在 temperature 0.0 下生成一个确定性输出，并用 temperature 0.7 额外采样 20 个候选点。尽管该方法探索了更广范围的合理位置，但会为这些模型带来显著计算开销。

相比之下，GUI-Actor 使用注意力图在单次前向传播中提出多个候选点。这带来高效得多的流程：与 AGUVIS 相比，**只需约 5% 的计算量**，同时在 ScreenSpot 和 ScreenSpot-v2 上取得明显更高的定位准确率，并在更具挑战的 ScreenSpot-Pro 基准上显著超过 AGUVIS。这些结果同时展示了 GUI-Actor 的效率，以及验证器在改进行动选择方面的通用有效性，即使接入其他模型也是如此。

H OSWorld 在线基准评测

为了评估所提出 GUI-Actor 在真实世界中的有效性，我们在 OS-World [16] 上使用 *GUI-Actor-7B* 进行快速验证。OS-World 是一个实时基准，用于测试真实桌面环境中的 GUI 智能体。我们关注一个包含 49 个 Windows 专用任务的精选子集，记为 OSWorld-

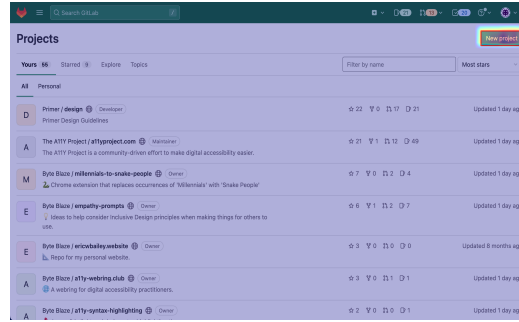
W，覆盖多种多步办公和多应用场景。每个任务都配有手写验证脚本，以保证可靠的自动评测。

遵循标准评测流水线，我们采用 GPT-4o 作为规划器。在每一步中，规划器观察当前 GUI 截图和用户指令，并生成自然语言计划。随后，该计划由底层定位模型通过基于坐标或无坐标机制定位为具体动作；定位模型在决定智能体成功与否时起关键作用。

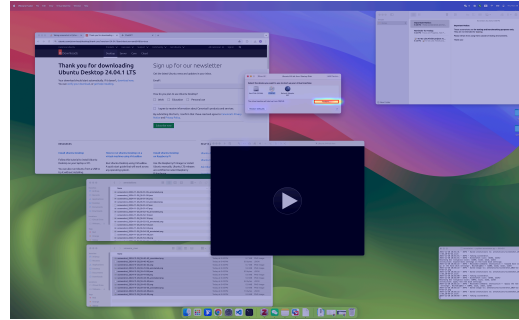
我们将 GUI-Actor 与若干最新视觉定位基线比较：Aguvis-7B [7]、NAVI [53] 和 Omni-Agent [54]。如表 10 所示，GUI-Actor 取得最高任务成功率 12.2%，超过 OmniAgent 和 NAVI (均为 10.2%)，并大幅超过 Aguvis-7B (4.0%)。这些结果凸显了 GUI-Actor 在复杂真实 GUI 环境中的有效性和鲁棒性。尽管训练时未接触 OSWorld-W 任务，GUI-Actor 仍能很好泛化到未见场景，比已有替代方案提供更准确且可靠的定位性能。

表 10: OSWorld-W 子集 (49 个实时 Windows GUI 任务) 上的任务成功率。GUI-Actor 在真实环境中显著超过已有定位模型。

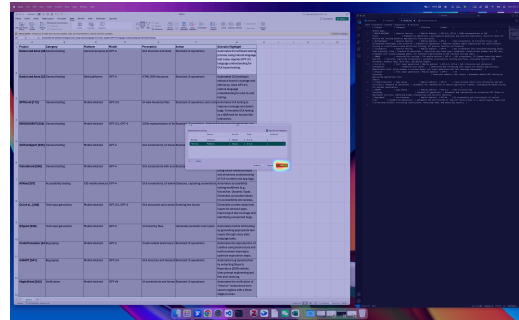
定位模型	成功率 (%)	完成任务数
Aguvis-7B (point sup.)	4.00	2/49
NAVI	10.2	5/49
OmniAgent	10.2	5/49
GUI-Actor-7B (Ours)	12.2	6/49



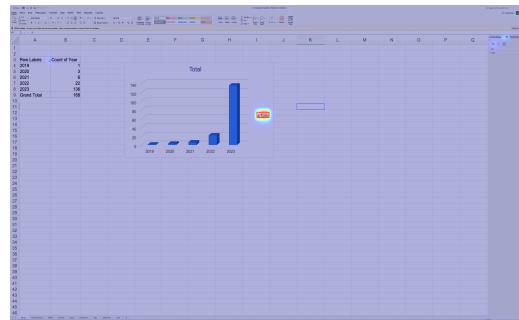
(a) ScreenSpot: "click the button to create a new project"



(b) ScreenSpot-Pro: "restart from CD"



(c) ScreenSpot-Pro: "confirm sort"



(d) ScreenSpot-Pro: "select the legend of the plot"

图 5: 来自 (a) ScreenSpot 和 (b)(c)(d) ScreenSpot-Pro 的可视化示例。每张图展示原始界面, 并叠加表示 GUI-Actor 兴趣区域的注意力图。注意力图与真实区域 (红色边界框) 大体重合, 说明模型能够有效捕获准确的 UI 元素。

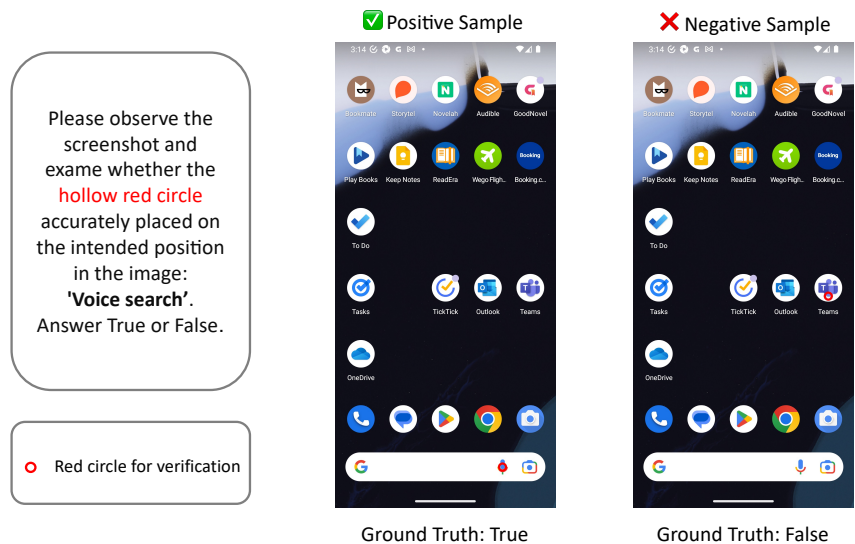


图 6: 用于训练定位验证器的正例和负例示意图。

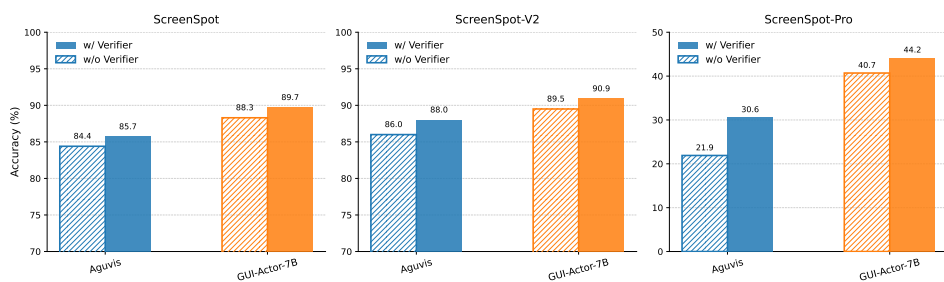


图 7: 使用验证器与 AGUVIS 比较。AGUVIS 需要进行 21 次推理以供验证。相比之下，GUI-Actor 只执行一次推理步骤，推理时只需约 5% 的计算量。